NTU

Assessment report

Mock-Up Test (Lab Test 01) 2026 Semester 1

aryan019

TOTAL SCORE

500/500

0 ATTEMPTED

14 of 14 questions

TEST BENCHMARK

Top 1%

Test time analysis

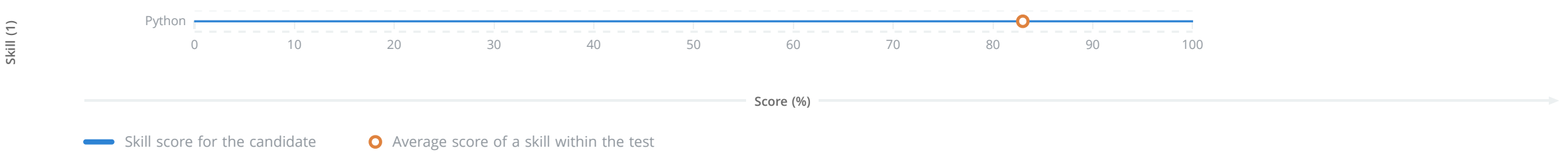
TEST INVITE TIME
Sep 13 2026, 07:46:39 PM +08 (Asia/Singapore)

TEST START TIME
Sep 24 2026, 12:59:36 AM +08 (Asia/Singapore)

TEST END TIME
Sep 24 2026, 02:36:16 AM +08 (Asia/Singapore)

TEST DURATION
1 hr 36 min 40 sec of 11 day used.

Skill-wise performance



Top Performer

Top performer: Python

You have scored the 1{ordinal} highest score in *Python*.

About aryan019

Languages
Python 3

Email ID
aryan019@e.ntu.edu.sg

Detailed submission report

Multiple Choice Questions

Questions attempted: 10 of 10

#	Questions (10)	Time (min)	Result	Score (100/100)
1	Which of the following situations requires Python’s cyclic garbage collector? **Options** — **Partial Scoring** A. Large string literals — No B. Variables going out of scope C. Two lists referencing each other CORRECTLY ATTEMPTED D. Adding an item to a list	00:32	✓	10/10

2	Consider the following binary tree:
	`10`
	`/ \`
	`5 20`
	`/ \ / \`
	`3 7 15 25`
	If this is a Binary Search Tree (BST), what will be the result of an inorder traversal?
		01:35	✓	10/10

	Options	Partial Scoring
	A. 10, 5, 3, 7, 20, 15, 25	No
	B. 3, 5, 7, 10, 15, 20, 25 CORRECTLY ATTEMPTED	
	C. 25, 20, 15, 10, 7, 5, 3	
	D. 7, 3, 5, 15, 20, 25, 10	

3	In the context of reversing a queue using a stack, what is the correct sequence of operations?	00:21	✓	10/10

Options	Partial Scoring
A. Dequeue from queue, push to stack, pop from stack, enqueue to queue CORRECTLY ATTEMPTED	No
B. Push to stack, pop from stack, enqueue to queue, dequeue from queue	
C. Enqueue to queue, push to stack, pop from stack, dequeue from queue	
D. Pop from stack, push to stack, enqueue to queue, dequeue from queue	

4	Which of the following is **NOT** an advantage of Python's automatic memory management?	01:30	✓	10/10

Options	Partial Scoring
A. Predictable timing of cleanup CORRECTLY ATTEMPTED	No
B. Prevents memory leaks	
C. No need for manual allocation/de-allocation	
D. Safer memory handling	

5	Which of the following is not an advantage of a singly linked list over arrays?	01:05	✓	10/10

Options	Partial Scoring
A. Dynamic memory allocation.	No
B. Efficient insertion at the beginning.	
C. Faster access to elements using index. CORRECTLY ATTEMPTED	
D. Memory usage is proportional to the number of elements.	

6	Which of the following statements about a Binary Search Tree is false?	00:28	✓	10/10

Options	Partial Scoring
A. The left child of a node contains values smaller than the node.	No
B. The right child of a node contains values larger than the node.	
C. An inorder traversal of BST always produces elements in ascending order.	
D. A postorder traversal of BST always produces elements in descending order. CORRECTLY ATTEMPTED	

7	What is a major advantage of a linked list over arrays?	00:44	✓	10/10

Options	Partial Scoring
A. Fixed size	No
B. Fast random access	
C. Dynamic memory allocation CORRECTLY ATTEMPTED	
D. Limited flexibility	

8	Which of the following scenarios is best suited for a queue rather than a stack?	00:59	✓	10/10

Options	Partial Scoring
A. Implementing function calls in recursion	No
B. Scheduling CPU processes in an operating system CORRECTLY ATTEMPTED	
C. Managing browser history	
D. None of the above	

9	Which of the following is not true about postfix expressions?	02:07	✓	10/10

Options	Partial Scoring
A. They do not require parentheses.	No
B. Evaluation is faster using a stack.	
C. Operators come before operands. CORRECTLY ATTEMPTED	
D. They can be easily evaluated from left to right.	

10	Suppose you insert the numbers 50, 30, 70, 20, 40, 60, 80 into an initially empty BST. What will be the height of the resulting tree? (Height of a leaf-node tree is 0)	00:52	✓	10/10

Options	Partial Scoring
A. 2 CORRECTLY ATTEMPTED	No
B. 3	
C. 4	
D. 5	

Programming Questions

Questions attempted: 4 of 4

#	Questions (4)	Time (min)	No. of attempts	Result	Score (400/400)
1	def reverseQueueSegment	22:26	2	✓	100/100

Given a queue and two indices, **m** and **n**, reverse the portion of the queue from **index m** to **index n** (**inclusive**) while keeping the rest unchanged.
Constraints:

- The queue is non-empty.
- The reversal must be performed using a **only one temporary stack.**
- You only use standard queue operations (enqueue, dequeue) and stack operations (push, pop).
- No additional data structures (such as lists or arrays) can be used.

Function Prototypedef reverseQueueSegment(queue, m, n):
Parameters:

- **queue**: A queue containing integers.
- **m**: The starting index of the segment to be reversed.
- **n**: The ending index of the segment to be reversed.

Returns: The modified queue.

Input Format

1. A space-separated list of integers representing the queue.
2. **Two integers m and n where 0≤m≤n< (length of the queue).**

Example 01:
input
Original Queue: 10 20 30 40 50 60 70
m = 2, n = 5
Output
Modified Queue: 10 20 60 50 40 30 70

Example 02
input
Original Queue: 1 2 3 4 5 6
m = 1, n = 3
OutputModified Queue: 1 4 3 2 5 6

Score	Result	Time	Time Complexity	Space Complexity	Memory	Language	Submitted on
100 BEST	✓	0.08504 sec	Unavailable	Unavailable	2 KB	Python 3	Sep 24 2026, 02:34:17 AM +08 (Asia/Singapore)

Source code Plaintext Link

```
class ListNode:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.size = 0
        self.head = None
        self.tail = None

    def find_node(self, index):
        if index < 0 or index >= self.size:
            return None
        temp = self.head
        for _ in range(index):
            temp = temp.next
        return temp

    def remove_node(self, index):
        if index < 0 or index >= self.size:
            return -1
        if index == 0:
            self.head = self.head.next
            if self.size == 1:
                self.tail = None
        else:
            prev = self.find_node(index - 1)
            prev.next = prev.next.next
            if index == self.size - 1:
                self.tail = prev
        self.size -= 1
        return 0

    def insert_node(self, index, value):
        if index < 0 or index > self.size:
            return -1
        new_node = ListNode(value)
        if index == 0:
            new_node.next = self.head
            self.head = new_node
            if self.size == 0:
                self.tail = new_node
        elif index == self.size:
            self.tail.next = new_node
            self.tail = new_node
        else:
            prev = self.find_node(index - 1)
            new_node.next = prev.next
            prev.next = new_node
        self.size += 1
        return 0

    def printList(self):
        cur = self.head
        if cur is None:
```

```
            print("Empty")
            return
        while cur is not None:
            print(cur.data, end=" ")
            cur = cur.next
        print(" ")

class Stack:
    def __init__(self):
        self.ll = LinkedList()

    def push(self, data):
        self.ll.insert_node(0, data)

    def pop(self):
        if self.isEmpty():
            return None
        data = self.ll.head.data
        self.ll.remove_node(0)
        return data

    def peek(self):
        if self.isEmpty():
            return None
        return self.ll.head.data

    def isEmpty(self):
        return self.ll.size == 0

    def get_size(self):
        return self.ll.size

class Queue:
    def __init__(self):
        self.ll = LinkedList()

    def enqueue(self, data):
        self.ll.insert_node(self.ll.size, data)

    def dequeue(self):
        if self.isEmpty():
            return None
        data = self.ll.head.data
        self.ll.remove_node(0)
        return data

    def getFront(self):
        if self.isEmpty():
            raise IndexError("Peek from empty queue")
        return self.ll.head.data

    def getSize(self):
        return self.ll.size

    def isEmpty(self):
        return self.ll.size == 0

    def printQueue(self):
        self.ll.printList()

def reverseQueueSegment(queue, m, n):
# Write your code here #
    #index m to n inclusive
    #non empty so dont need check, unless m and n fall out of size, also dont need because main checks already
    #only 1 temporary stack
    #no list or arrays
    #start at 0 and end at size-1

    #likely dequaeue and enqueu  so not reversed
    #reversed the indexes using the stack

    size = queue.getSize()
    stack = Stack()

    for _ in range(m):
        queue.enqueue(queue.dequeue())

    for _ in range(n - m + 1):
        stack.push(queue.dequeue())

    while not stack.isEmpty():
        queue.enqueue(stack.pop())

    for _ in range(size - n - 1):
         queue.enqueue(queue.dequeue())



if __name__ == "__main__":
    queue = Queue()

    print("Enter a list of numbers, terminated by any non-digit character: ", end="")
    input_string = input()
    numbers = input_string.split()
```

```
152
153     for num in numbers:
154         try:
155             queue.enqueue(int(num))
156         except ValueError:
157             break
158
159     m, n = map(int, input("Enter indices m and n separated by space: ").split())
160
161     print("Original Queue:", end=" ")
162     queue.printQueue()
163
164     # Validate input indices before calling the function
165     if m < 0 or n >= queue.getSize() or m > n:
166         print("Invalid indices")
167     else:
168         reverseQueueSegment(queue, m, n)
169         print("Modified Queue:", end=" ")
170         queue.printQueue()
```

2 def in2postS(expr): 34:42 10 100/100

Write a function, **in2postS()** to convert an infix expression into a postfix expression stack. We assumed that the character string only contains positive integer numbers, operators ('**', '+', '-', '*' and '/') and parentheses.

Function Prototype

def in2postS(expr):

Example 01

Sample Input: 3 + 4 * 5

Sample Output: 3 4 5 * +

Example 02

Sample Input: (1 + (2 ** 3) * (4 / (5 - 6)))

Sample Output: 1 2 3 ** 4 5 6 - / * +

Example 03

Sample Input: ((2 + 3) * (4 - 1)) / (5 + (6 * (7 - 8)))

Sample Output: 2 3 + 4 1 - * 5 6 7 8 - * + /

Score	Result	Time	Time Complexity	Space Complexity	Memory	Language	Submitted on
100 BEST	✓	0.11915 sec	Unavailable	Unavailable	2 KB	Python 3	Sep 24 2026, 02:34:17 AM +08 (Asia/Singapore)

Source code Plaintext Link

```
1   # Define operator precedence globally
2   PRECEDENCE = {'+': 1, '-': 1, '*': 2, '/': 2, '**': 3}
3
4   class Node:
5       def __init__(self, data):
6           self.data = data
7           self.next = None
8
9   class Stack:
10      def __init__(self):
11          self.top = None
12          self.size = 0
13
14      def isEmpty(self):
15          return self.size == 0
16
17      def push(self, data):
18          new_node = Node(data)
19          new_node.next = self.top
20          self.top = new_node
21          self.size += 1
22
23      def pop(self):
24          if self.isEmpty():
25              raise IndexError("Pop from empty stack")
26          value = self.top.data
27          self.top = self.top.next
28          self.size -= 1
29          return value
30
31      def peek(self):
32          if self.isEmpty():
33              raise IndexError("Peek from empty stack")
34          return self.top.data
35  def in2postS(expr):
36  # Write your code here #
37      output = Stack()
38      operator = Stack()
39
40      for i in expr:
41          if i == '(':
42              operator.push(i)
43          elif i == ')':
```

```
            while not operator.isEmpty() and operator.peek() != '(':
                output.push(operator.pop())
            if not operator.isEmpty() and operator.peek() == '(':
                operator.pop()
        elif i.isalnum():
            output.push(i)
        else:
            while (not operator.isEmpty() and operator.peek() != '(' and (PRECEDENCE[i] <= PRECEDENCE[operator.peek()]
            or (PRECEDENCE[i] == PRECEDENCE[operator.peek()] and i != '**'))):
                output.push(operator.pop())
            operator.push(i)

    while not operator.isEmpty():
        if operator.peek() == '(':
            operator.pop()
        else:
            output.push(operator.pop())

    return output


if __name__ == "__main__":
    infix = input("Enter infix expression (space separated): ")
    postfix = in2postS(list(infix.split(' ')))

    # Reverse stack to print in correct postfix order
    temp = Stack()
    while not postfix.isEmpty():
        temp.push(postfix.pop())

    result = ""
    while not temp.isEmpty():
        result += temp.pop() + " "

    print("Postfix expression:", result.strip())
```

3 def findUnion(head1, head2): 23:20 5 100/100

Given two **sorted linked lists**, implement a function to create a new linked list that represents the **union** of the two original lists. The new linked list should be created without modifying the original lists.

Constraints:

1. The linked lists are sorted in **non-decreasing order** (ascending order, allowing duplicates).
2. The elements in the linked lists are **not necessarily distinct** (duplicates are allowed in the input lists).
3. The **new linked list should contain all unique elements** from both input linked lists.
4. The **order of elements in the union linked list must be the same** as they appear in the original lists (sorted order).
5. **Additional data structures such as lists, dictionaries, sets, or arrays are not allowed**.

Function Prototype:

def findUnion(head1, head2):

Input Format:

- Two space-separated lists of integers representing the two sorted linked lists.
- Each linked list is terminated by a non-numeric character.

Output Format:

- A space-separated list of integers representing the **union** of the two linked lists.

Example 1 (Basic Case with Duplicates in Input)

Input:

list1 = 1 -> 2 -> 2 -> 3 -> None

list2 = 2 -> 3 -> 4 -> None

Output:

Union: 1 2 3 4

Example 2 (Both Lists Have Unique Elements)

Input

list1 = 1 -> 3 -> 5 -> 7 -> None

list2 = 2 -> 4 -> 6 -> 8 -> None

Output

Union: 1 2 3 4 5 6 7 8

Example 3 (One List Is a Subset of Another)

Input:

list1 =3 ->3 ->4-> 4 ->5 ->5-> 6-> 6-> None

list2 =1-> 1-> 2-> 2-> 3-> 3 ->4 ->4 ->5-> 5 ->6 ->6-> None

output

Union: 1 2 3 4 5 6

Score	Result	Time	Time Complexity	Space Complexity	Memory	Language	Submitted on
100 BEST	✓	0.10204 sec	Unavailable	Unavailable	2 KB	Python 3	Sep 24 2026, 02:34:17 AM +08 (Asia/Singapore)

Source code Plaintext Link

```
class Node:
    def __init__(self, data):
        self.data = data
```

```
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None
        self.size = 0

    def findNode(self, index):
        if index < 0 or index >= self.size:
            raise ValueError("Invalid position")
        if self.head is None:
            raise ValueError("List is empty")

        cur = self.head
        while index > 0:
            cur = cur.next
            index -= 1
        return cur

    def insertNode(self, data, index):
        if index < 0 or index > self.size:
            raise ValueError("Invalid position")

        new_node = Node(data)

        if index == 0:
            new_node.next = self.head
            self.head = new_node
            self.size += 1
            return True

        prev_node = self.findNode(index - 1)
        if prev_node is not None:
            new_node.next = prev_node.next
            prev_node.next = new_node
            self.size += 1
            return True
        return False

    def removeNode(self, index):
        if index < 0 or index >= self.size:
            raise ValueError("Invalid position")

        if self.head is None:
            return False

        if index == 0:
            cur = self.head
            self.head = cur.next
            self.size -= 1
            return True

        pre = self.findNode(index - 1)
        if pre is not None and pre.next is not None:
            cur = pre.next
            pre.next = cur.next
            self.size -= 1
            return True
        return False

    def printList(self):
        cur = self.head
        if cur is None:
            print("Empty")
            return
        while cur is not None:
            print(cur.data, end=" -> ")
            cur = cur.next
        print("None")
def findUnion(head1, head2):
# Write your code here #
    #duplicates allowed, ascending
    #contian all unique elemnets
    #acednig sorted
    #no other data

    #create dummy node
    dummy = Node(0)

    l1 = head1
    l2 = head2
    cur = dummy

    while l1 and l2:
        if cur.data == l1.data:
            l1 = l1.next
        elif cur.data == l2.data:
            l2 = l2.next
        elif l1.data <= l2.data:
            cur.next = Node(l1.data)
            l1 = l1.next
            cur = cur.next
        else:
            cur.next = Node(l2.data)
```

```
            cur = cur.next

    if l1:
        while l1:
            if l1.data == cur.data:
                l1 = l1.next
            else:
                cur.next = Node(l1.data)
                l1 = l1.next
                cur = cur.next
    elif l2:
        while l2:
            if cur.data == l2.data:
                l2 = l2.next
            else:
                cur.next = Node(l2.data)
                l2 = l2.next
                cur = cur.next

    return dummy.next



if __name__ == "__main__":
    # Helper function to create a linked list from a string of space-separated numbers
    def create_linked_list_from_input(input_string):
        linked_list = LinkedList()
        numbers = []
        for item in input_string.split():
            try:
                numbers.append(int(item))
            except ValueError:
                continue

        for i, num in enumerate(numbers):
            linked_list.insertNode(num, i)

        return linked_list

    # Read input from file (or stdin if redirect is used)
    try:
        # Read first line for first linked list
        input_str1 = input()
        list1 = create_linked_list_from_input(input_str1)

        # Read second line for second linked list
        input_str2 = input()
        list2 = create_linked_list_from_input(input_str2)

        # Find the union
        result = findUnion(list1.head, list2.head)

        # Print the union list
        if result is None:
            print("No elements")
        else:
            temp = result
            while temp:
                print(temp.data, end=" ")
                temp = temp.next
            print()

    except EOFError:
        print("Error: Input file should contain at least two lines with the linked lists data.")
```

4 def sumOfLeafNodes(root): 05:53 3 100/100

Given a **Binary Search Tree (BST)**, implement a function to find and return the **sum of all leaf nodes**. A **non-leaf node** is any node that has no children.

Constraints:

- The BST is **non-empty**.
- The BST follows the standard **binary search tree** property:
  - The left subtree contains only nodes with values **less than** the root.
  - The right subtree contains only nodes with values **greater than** the root.
- The function must use **recursion** to compute the sum of non-leaf nodes.

Function Prototype:def sumOfLeafNodes(root):

Input Format:

1. The BST is built from user input.
2. The user enters **multiple integers**, and each integer is inserted into the BST.
3. The function sumOfLeafNodes(root) is called to compute the sum.

Output Format:

An **integer** representing the sum of all leaf nodes.

Example 1:

Input (BST Structure):

```
      10
     /  \
    5    20
   / \    \
```

3 7 30

Output

Sum of leaf nodes: 3 + 7 + 30 = 40

Example 2:

Input (BST Structure):

```
      15
     /  \
    10   25
   /  \    \
  5   12    30
```

OutputSum of non-leaf nodes: 5 + 12 + 30 = 47

Score	Result	Time	Time Complexity	Space Complexity	Memory	Language	Submitted on
100 BEST	✓	0.08502 sec	O(n)	O(n)	2 KB	Python 3	Sep 24 2026, 02:34:17 AM +08 (Asia/Singapore)

Source code Plaintext Link

```
class BSTNode:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

def insert(root, data):
    """ Inserts a node into the BST. """
    if root is None:
        return BSTNode(data)

    if data < root.data:
        root.left = insert(root.left, data)
    else:
        root.right = insert(root.right, data)

    return root

def inorderRecursive(root):
    """ Prints in-order traversal of the BST (Left, Root, Right). """
    if root:
        inorderRecursive(root.left)
        print(root.data, end=" ")
        inorderRecursive(root.right)
def sumOfLeafNodes(root):
# Write your code here #
    #mo need check for empty
    #must use recusion
    if root is None:
        return 0
    if root.left or root.right:
        return sumOfLeafNodes(root.left) + sumOfLeafNodes(root.right)
    else:
        return root.data

if __name__ == "__main__":
    root = None
    print("1: Insert an integer into BST")
    print("2: Sum of all leaf nodes")
    print("0: Quit")

    while True:
        choice = int(input("\nPlease input your choice (1/2/0): "))

        if choice == 1:
            value = int(input("Input an integer to insert: "))
            root = insert(root, value)
            # Automatically print in-order traversal after each insertion
            print("In-order traversal: ", end="")
            inorderRecursive(root)
            print()
        elif choice == 2:
            print(f"Sum of leaf nodes: {sumOfLeafNodes(root)}")
        elif choice == 0:
            break
        else:
            print("Invalid choice. Please enter 1, 2, or 0.")
```

* - This information is based on the data that is available untill Sep 24 2026, 02:40:48 AM +08 (Asia/Singapore). Your rank may be updated once all the candidates have successfully completed their tests.

Thank you for taking this test. All the best for further processes.

Note: This report is generated by HackerEarth. To know more, visit www.hackerearth.com